Cryptography and Computer System Security
Cairo University, Faculty of Engineering
Computer Engineering Dept.
CMPS426

Cairo University

Security Course Project

Secure Password Manager + Capture The Flags

Component	Description
Part 1	Secure Password Manager (4 Modules)
Part 2	6 CTF Challenges

Part 1: Secure Password Manager

Description

Password managers are one of the most practical applications of cryptography in everyday life. They allow users to securely store, retrieve, and manage credentials using strong encryption, so that a single master password protects hundreds of accounts.

In this project, you will build a **Secure Password Manager** — a command-line application that uses AES for encryption, SHA for hashing, ElGamal Digital Signatures for integrity verification, and Diffie-Hellman for secure vault (a **secure encrypted storage**) export between users.

You need to implement Diffie-Hellman, ElGamal Digital Signatures, and the overall system logic from scratch. **You may import ready-made implementations for AES (used internally for vault encryption) and SHA hash functions.**

System Overview

Local workflow: A user sets up a vault with a master password, stores credentials, and retrieves them. Every vault operation is integrity-checked with digital signatures.

Export workflow: A user exports their vault to another user/device using Diffie-Hellman key exchange, with digital signatures protecting against man-in-the-middle attacks.

Module 1: ElGamal Key Management

Inputs:

➔ User identity (username)

Outputs:

➔ ElGamal public/private key pair

➔ Exported public key file (for sharing)

Functionality:

When a user initializes their password manager for the first time, this module generates EL Gamal key pair (for digital signatures in Modules 3 and 4):

- Read shared parameters (large prime p and primitive root α) from a configuration file, or generate them per user.
- Generate an ElGamal key pair using these parameters.

Properties:

1. ElGamal must be implemented from scratch.

2. **Key storage:**
   - ➢ The private key is saved locally and protected.
   - ➢ The Public key is exportable for sharing with other users.
   - ➢ The ElGamal public key allows others to verify this user's signatures.

3. **Private keys must never be exposed or transmitted**

4. The key pair is generated once during user initialization and reused across all modules.

Module 2: Vault Encryption & Credential Management

Inputs:

- Master password
- Credential operations (add / retrieve / update / delete)

Outputs:

- Encrypted vault file
- Decrypted credentials on retrieval (the vault is decrypted in memory and the selected entry is displayed to the user)

Functionality:

During vault initialization, the **SHA-256** of the master password is used as the AES **data key.** This data key is used to encrypt and decrypt the entire vault file.

On adding:

- The user provides the master password.
- The entire vault is decrypted using the **data key**.
- The new credential (website, username, password) is added.
- The entire vault is re-encrypted using the data key.
- The vault is then re-signed (see Module 3).

On retrieval:

- The user provides the master password.
- The entire vault is decrypted in memory using the **data key**.
- The selected entry is displayed to the user.

On update/delete:

- The user provides the master password.
- The entire vault is decrypted using the data key.
- The selected entry is modified/deleted.
- The entire vault is re-encrypted using the data key.
- The vault is then re-signed.

This approach uses symmetric encryption for both security and efficiency, allowing encryption of arbitrary-length data while ensuring that only a user with the correct master password can access the vault contents.

The vault is stored as a **JSON file**. The JSON structure contains:

- The AES-encrypted credential entries
- The digital signature over the vault contents (see Module 3).

JSON file example:

```
{
  "encrypted_vault": "a3f8b2c1d4e5...(AES-encrypted credentials)",
  "signature": "7b9f2a4c8d1e...(ElGamal digital signature)"
}
```

Properties:

1. Adding, retrieving, updating, and deleting credentials should work correctly.
2. The vault file should be unusable without the correct master password.
3. You are free to import a ready-made implementation for AES and SHA-256.
4. Note: you should use AES_GCM Mode `AES.new(key, AES.MODE_GCM)`

Module 3: Digital Signatures for Vault Integrity

Inputs:

- Vault file to sign or verify
- User's ElGamal private key from Module 1 (for signing) or ElGamal public key from Module 1(for verification)

Outputs:

- Signed vault file
- Verification result (valid / invalid + alert)

Functionality

Every time the vault is modified (credential added, updated, or deleted), the vault is re-signed using the user's private key with an **ElGamal digital signature.**

Every time the vault is opened, the signature is verified before any credentials are shown. If the signature is invalid (i.e., someone tampered with the vault file), the user is alerted and the vault refuses to open.

1. **Signing process:**
    a. Compute a SHA-256 hash of the vault contents (encrypted content).
    b. Sign the hash using the user's private key (ElGamal signature)
    c. Store the signature alongside the vault.
2. **Verification process:**
    a. Compute the SHA-256 hash of the current vault contents (encrypted content)
    b. Verify the stored signature against the hash using the user's ElGamal public key.
    c. If verification fails, refuse to open and warn the user.

Properties:

1. The digital signature scheme must be implemented from scratch.
2. You are free to import a ready-made function for **SHA-256.**
3. Any manual edit to the vault file (even a single bit flip) must cause verification to fail.
4. This module does not generate any keys — it relies entirely on the ElGamal key pair generated by Module 1.

Module 4: Secure Vault Export via Diffie-Hellman

Inputs:

- Device 1's vault
- Device 1's ELGamal private key
- Device 2's ELGamal public key
- Device 1's ElGamal public key (shared with recipient for signature verification)
- Shared Diffie-Hellman (DH) parameters (q, α from a public config file)

Outputs:

- Encrypted and signed vault export package
- Successfully imported vault on recipient's side

Functionality:

This module enables a user to securely export their vault to another device.

Key Exchange Phase

1. Both devices read the shared Diffie-Hellman parameters (q,α), where q is a large prime and α is a primitive root modulo q.
2. Each device generates a Diffie-Hellman key pair: a private key and a public key.
3. Device 1 sends its DH public key, signed with its digital signature.
4. Device 2 verifies the signature; if invalid, the process is aborted.
5. Device 2 sends its DH public key, also signed.
6. Device 1 verifies the signature; if invalid, the process is aborted.
7. Both devices compute the shared secret.
8. A session AES-256 key is derived from the shared secret using SHA-256.

Transfer Phase

1. On Device 1, the user enters the master password.
2. The AES key (data key) is used to **decrypt all vault entries locally.**
3. The decrypted vault data (in memory only) is then **encrypted with the session key derived from Diffie-Hellman.**
4. A new ElGamal digital signature is computed over the session-key-encrypted data using Device 1's private key.
5. The encrypted data along with the new signature is transmitted to Device 2.
6. Device 2 verifies the new signature using Device 1's ElGamal public key to ensure the data has not been tampered with in transit.
7. If verification fails, the import is aborted.

Import Phase

1. Device 2 derives the same session key from the shared secret.
2. The received encrypted vault data is decrypted using this session key.
3. The user enters their master password (same or new).
4. A new AES key (data key) is derived from the master password, and **the vault is re-encrypted and stored locally.**
5. A new ElGamal digital signature is computed over the newly encrypted vault using Device 2's private key, replacing Device 1's signature.

Properties:

1. Diffie-Hellman must be implemented from scratch.
2. If either signature verification fails, the export must abort.
3. You are free to import a ready-made implementation for AES and SHA-256.
4. DH keys are ephemeral (generated per export session), while signature keys are long-lived

Bonus:

- Implementing a UI instead of a CLI will be considered a bonus within the project's total grade and will not be propagated to any other classwork grades.

Part 1 Deliverables:

1. A program implementing all 4 modules with a working CLI interface.
2. Should demonstrate the full workflow: setup → store credentials → sign → verify →export to another user → import.
3. README file explaining how to run your application.
4. Documentation covering your design decisions, algorithm choices, and any challenges faced.

Part 2: CTFs (Capture The Flags)

In this part you will solve 9 CTF tasks. You can find the challenge files attached with this document.

Each task will ask you to find a hidden flag in a different way and your job is to find this flag correctly. You may deal with text files, images, audio, network captures, log files, or binary data.

The flag structure can be one of:

1. CMPN{some_text}
2. FLAG{some_text}
3. SOME_TEXT

CTF 1 — Packet Analysis

You have been given a network capture file traffic.pcapng. A secret flag was transmitted during a suspicious TCP session.

The flag has been split across multiple TCP packets and base64-encoded before transmission. Reassemble the data from the TCP stream and decode it to find the original flag.

> *Hint: Filter for TCP traffic on port 4444. Extract the payload data from each packet in order and concatenate before decoding.*

CTF 2 — Image Manipulation

You have two PNG images: layer1.png and layer2.png. Each image appears to be random noise on its own, but together they reveal a secret.

CTF 3 — Bit Shifting

You are given a text file shifted.txt containing a sequence of decimal numbers. Each number represents a character that has been bit-shifted. Find the right operation to recover the original ASCII characters and reveal the flag.

```
shifted.txt contents:

140 152 148 142 173 137 150 137 141 156
164 137 163 145 145 137 155 145 137 142
165 164 137 171 157 165 137 143 141 156
137 163 150 151 146 164 137 155 145 175
```

CTF 4 — Steganography

Steganography is the practice of hiding data inside another file. You are given an image file innocent.png that looks like an ordinary photograph. Something is HIDING inside this image using LSB (Least Significant Bit) steganography.

CTF 5 — CBC Padding Oracle

You are testing a website that is encrypting a secret. However, the website is designed badly and leaks whether after decryption the padding of clear text is right or gives an error. Can you use this piece of information to leak the secret without knowing the key?

http://cbc-ctf.westeurope.azurecontainer.io:5000/

CTF 6 — RSA Key Recovery

You intercepted an encrypted message along with the RSA public key used to encrypt it. However, the key was generated poorly — the primes used are too close together.

Public Key:

```
n = 640646482447890645817038420418465055769199395196881740852277977 51
e = 65537
```

Ciphertext (decimal):

```
c = 384652930285732750173698465054989404926380486671305958288691052 99
```

CTF Deliverables:

- Submit a single **zip file** containing:
    - A **PDF document** containing your CTF solutions for Part 2
    - Any code files used to solve CTF challenges

Rules

1. Group size is up to **4 members**.
2. The deadline is **Week 13**
3. The project should be implemented in **Python**.
4. Diffie-Hellman, and ElGamal Digital Signatures must be implemented **from scratch**. Using external cryptography libraries for these algorithms will result in a **zero grade** for the affected module. You may only import ready-made implementations for AES and SHA hash functions as stated in the project description.
5. **AI-generated/copied code is strictly prohibited.** Any submission found to contain code generated by AI tools will receive a **zero grade for the entire project**.
6. **Copying from colleagues is plagiarism.** Submissions will be checked for similarity against each other. Teams found to have shared or copied code will **all receive a zero grade**. This applies to both the team that copied and the team that provided the code.
7. **CTF solutions without explanation will be discarded.** For each CTF, you must document your thought process, the steps you followed, the tools you used, and how you arrived at the flag.
8. Any code used to solve CTF problems must be included in the submission.